

Tomasulo-Simulator

Moritz Schubert

June 22, 2026

1 Schriftliche Beschreibung der internen Struktur und Funktionsweise des Tomasulo-Simulators

Der entwickelte Simulator implementiert eine vereinfachte Version des Tomasulo-Algorithmus zur dynamischen Befehlsausführung in Prozessoren. Ziel des Projekts ist es, Datenabhängigkeiten aufzulösen, Out-of-Order-Execution zu ermöglichen und die Auswirkungen verschiedener Latenzen auf die Programmausführung sichtbar zu machen. Der Simulator wurde in C++ umgesetzt.

Die Architektur des Simulators besteht aus mehreren zentralen Komponenten: einem Registersatz, einem Hauptspeicher, Reservation Stations (RS), zwei verschiedenen funktionalen Einheiten (Functional Units), wovon eine für LOAD/STORE und eine für arithmetische Operationen zuständig ist, einem Common Data Bus (CDB) sowie einer Instruktionwarteschlange. Zusätzlich werden interne Verwaltungsvariablen verwendet, um Latenzen, aktuelle Zustände und Statistikdaten zu speichern. Der Ablauf simuliert die fünf verschiedenen Phasen einer RISC-V Architektur, inklusive der Berücksichtigung von Warmupaktakten.

Die Befehle eines Programms werden zunächst aus einer Textdatei, welche bei Ausführung des Programms in der Kommandozeile eingegeben werden kann, eingelesen und in die `instructionQueue` gespeichert. Unterstützt werden arithmetische Operationen (ADD, SUB, MUL, DIV) sowie Speicheroperationen (LOAD, STORE). Zusätzlich existieren interne Kontrollbefehle wie NOP, END, welche automatisch vom Simulator verwendet werden und nicht manuell eingesetzt werden sollten oder Befehle zur Ausgabe von Statistiken. Letztere sind: `v` — Zeilenweise Erklärung ein-/ausschalten (standardmäßig aus) `s` — Aktuellen Zustand aller Reservierungsstationen und Register ausgeben `h` — IPC und Gesamt-Zyklenanzahl bis zu diesem Punkt ausgeben `i` — Anzahl der bisherigen Stall-Zyklen und aufgelösten Abhängigkeiten ausgeben Sollte bei der Ausführung des Programms die Fehlermeldung "An ERROR occured while reading programm" auftreten, bedeutet das, dass eine Zeile des Programms nicht der erwarteten Syntax entspricht. Dies wird nicht zum Absturz führen, allerdings wird die Zeile ignoriert und mit einem NOP ersetzt werden. Nach Ausführung des Programms, wird der Nutzer geben, den Namen der Programmdatei und die gewünschten Cycles anzugeben. Die Anzahl der angegebenen Cycles gibt an, wie weit das Programm in diesem Schritt fortschreiten soll. Alle relevanten Parameter können in den Defines am Kopf des Codes oder über Compiler-Flags mit `-DParameter=0` beim Kompilieren angepasst werden.

Der Registersatz (`Register reg[]`) speichert neben dem eigentlichen Registerwert auch Informationen darüber, ob das Register aktuell von einer Reservation Station beschrieben

wird. Dafür besitzt jedes Register das Feld **producerRS**, welches auf die Reservation Station verweist, die das nächste gültige Ergebnis für dieses Register erzeugen wird. Dadurch wird Register Renaming realisiert. WAW- und WAR-Abhängigkeiten werden auf diese Weise vermieden, da immer nur die aktuellste schreibende Reservation Station im Register eingetragen ist.

Für die Befehlsausführung existieren zwei getrennte Gruppen von Reservation Stations. Die **alu_rs[]** dienen für arithmetische Operationen, während **ls_rs[]** für Speicherzugriffe verwendet werden. Jede Reservation Station enthält Informationen über die Operation, Zielregister, Operandenwerte und mögliche Abhängigkeiten zu anderen Reservation Stations. Falls ein benötigter Operand noch nicht verfügbar ist, wird statt des Wertes ein Verweis auf die produzierende Reservation Station gespeichert (**Rs1**, **Rs2** bzw. **dstRs**). Dadurch können Befehle bereits ausgegeben werden, obwohl ihre Operanden noch fehlen.

Die Ausgabe neuer Instruktionen erfolgt in der Funktion **instructionFetchDecode()**. Dort wird zunächst anhand des Opcodes entschieden, welche Art von Reservation Station benötigt wird. Anschließend wird eine freie Reservation Station gesucht. Ist keine freie Reservation Station vorhanden, entsteht ein struktureller Stall, welcher durch den Zähler **stallCycles** erfasst wird. Nach erfolgreicher Reservierung werden die Operanden überprüft. Bereits verfügbare Werte werden direkt übernommen, während bei noch laufenden Berechnungen nur die entsprechende Reservation Station gespeichert wird.

Die eigentliche Befehlsausführung erfolgt in der Funktion **execute()**. Dabei werden die Reservation Stations zyklisch durchsucht, wodurch gegebenenfalls Befehle vorgezogen werden können. Sobald alle Operanden eines Befehls verfügbar sind, kann die entsprechende funktionale Einheit die Ausführung starten. Der Simulator besitzt getrennte funktionale Einheiten für ALU-Operationen und Speicheroperationen. Die Latenzen der einzelnen Operationen sind über Konstanten konfigurierbar (**Latency_AddSub**, **Latency_Mul**, **Latency_Div**, **Latency_LS**). Während der Ausführung blockiert die funktionale Einheit für die entsprechende Anzahl an Takten.

Sobald eine Operation abgeschlossen ist, wird das Ergebnis über den Common Data Bus (**cdb[]**) verteilt. Der CDB dient als Broadcast-System: Alle Reservation Stations prüfen, ob die ausgesendeten Ergebnisse zu ihren wartenden Operanden passen. Falls dies der Fall ist, werden die fehlenden Werte übernommen und die Abhängigkeiten entfernt. Dadurch können Befehle sofort nach Verfügbarkeit ihrer Operanden weiter ausgeführt werden. Dieses Verhalten bildet den Kern des Tomasulo-Algorithmus nach.

Die Rückschreibphase erfolgt in **writeBack()**. Dort werden die Werte aus dem CDB in die Registerdatei geschrieben, sofern die Reservation Station weiterhin als aktueller Produzent des Registers eingetragen ist. Dadurch wird verhindert, dass ältere Ergebnisse neuere überschreiben. Nach erfolgreichem Writeback wird die entsprechende Reservation Station wieder freigegeben.

Der Simulator unterstützt außerdem die Erkennung von RAW-Abhängigkeiten (Read After Write). Muss ein Befehl auf das Ergebnis einer vorherigen Instruktion warten, wird dies intern gezählt (**rawPrevented**). Tomasulo verhindert dabei globale Pipeline-Stalls, indem unabhängige Instruktionen weiterhin ausgeführt werden können. Ein echter Stall entsteht nur dann, wenn keine ausführbare Instruktion mehr vorhanden ist oder keine freie Reservation Station verfügbar ist. In letzterem Fall wird für jeden eigentlich bereiten Befehl ein Stall gezählt. Hierbei werden die beiden Befehle, welche in der Warmupphase fetched aber noch nicht executed werden, nicht als Stalls mitgezählt. Das liegt daran, dass erst ab Takt 3 ein **execute()** ausgeführt wird und ansonsten in jedem Takt die beiden

extra Befehle als Stalls gezählt werden würden.

Zusätzlich werden verschiedene Leistungsdaten berechnet und ausgegeben. Dazu gehören die Anzahl ausgeführter ALU- und Speicherbefehle, die Anzahl der Stallzyklen, die Anzahl veränderter RAW-Abhängigkeiten sowie die durchschnittliche IPC (Instructions per Cycle). Dadurch kann das Verhalten des Simulators analysiert und mit unterschiedlichen Programmen getestet werden.

2 Validierung

2.1 Keine Abhängigkeiten

```
1      V
2      LOAD R0, 0(R1)
3      SUB R2, R3, R3
4      MUL R4, R5, R5
5      DIV R6, R7, R7
6      SUB R0, R2, R1
```

Das zu erwartende Ergebnis mit der vorgegebenen Standardkonfiguration enthält: 13 Takte, 8 Stalls aufgrund von MUL und DIV, eine IPC von 0.384615 (5/13), keine gelösten RAW-Abhängigkeiten, 17 Prozent LS-Auslastung und 83 Prozent ALU-Auslastung. Außerdem läuft das LOAD parallel zu den arithmetischen Befehlen. Mein Simulator erzeugt folgendes Ergebnis:

```
Cycles: 13 IPC: 0.923077 Stalls: 8 Structural stalls: 0 RAW prevented: 0 Auslastung ALU: 83.3333% Auslastung L/S: 16.6667%
```

2.2 RAW

```
1      V
2      DIV R0, R1, R1
3      ADD R2, R0, R0
```

Das zu erwartende Ergebnis mit der vorgegebenen Standardkonfiguration enthält: 9 Takte, 4 Stalls aufgrund von DIV, eine IPC von 0.22 (2/9), zwei gelösten RAW-Abhängigkeiten und 100 Prozent ALU-Auslastung. Mein Simulator erzeugt folgendes Ergebnis:

```
Cycles: 9 IPC: 0.222222 Stalls: 4 Structural stalls: 0 RAW prevented: 2 Auslastung ALU: 100% Auslastung L/S: 0%
```

2.3 Struktureller Stall

```
1      V
2      DIV R0, R1, R1
3      DIV R1, R0, R0
4      DIV R0, R1, R1
5      DIV R1, R0, R0
6      DIV R0, R1, R1
7      DIV R1, R0, R0
8      DIV R0, R1, R1
9      DIV R1, R0, R0
```

Das zu erwartende Ergebnis mit der vorgegebenen Standardkonfiguration enthält: 43 Takte, 84 Stalls aufgrund von wartenden DIVs und 15 strukturelle Stalls wegen einer oft vollen RS (Standardkonfiguration: 4 AluRs-Slots), eine IPC von 0.186047 (8/43), keinen gelösten RAW-Abhängigkeiten und 100 Prozent ALU-Auslastung. Mein Simulator erzeugt folgendes Ergebnis:

Cycles: 43 IPC: 0.186047 Stalls: 84 Structural stalls: 15 RAW prevented: 14 Auslastung ALU: 100% Auslastung L/S: 0%

2.4 WAR und WAW

```

1      V
2      LOAD R0, 0(R1)
3      ADD R0, R1, R1 (WAW)
4      SUB R1, R2, R2 (WAR)

```

Das zu erwartende Ergebnis mit der vorgegebenen Standardkonfiguration enthält: 5 Takte, 0 Stalls, eine IPC von 0.6 (3/5), keine gelösten RAW-Abhängigkeiten, 50 Prozent ALU-Auslastung und 50 Prozent LS-Auslastung. Mein Simulator erzeugt folgendes Ergebnis:

Cycles: 5 IPC: 0.6 Stalls: 0 Structural stalls: 0 RAW prevented: 0 Auslastung ALU: 50% Auslastung L/S: 50%

2.5 LOAD und STORE

```

1      V
2      LOAD R0, 0(R1)
3      STORE R2, 0(R3)

```

Das zu erwartende Ergebnis mit der vorgegebenen Standardkonfiguration enthält: 7 Takte, 1 Stall aufgrund von LOAD, eine IPC von 0.285714 (2/7), keine gelösten RAW-Abhängigkeiten und 100 Prozent LS-Auslastung. Mein Simulator erzeugt folgendes Ergebnis:

Cycles: 7 IPC: 0.285714 Stalls: 1 Structural stalls: 0 RAW prevented: 0 Auslastung ALU: 0% Auslastung L/S: 100%

Figure 1:

3 Experimente

3.1 a. Vergleich mit In-Order-Pipeline ohne dynamisches Scheduling

Scriptname	Takte	IPC	Erklärung
NoDependency.txt	13	0.384615	Kein Unterschied zu Tomasulo
RAW.txt	10	0.222	Hier wird ein Cycle mehr benötigt, da Tomasulo die RAW Abhängigkeit durch Forwarding lösen kann und somit nicht auf das Writeback warten muss
StructuralStall.txt	50	0.16	Auch hier wird pro Instruction, außer der ersten, ein Cycle mehr benötigt, da in diesem Script jede Instruction von der Vorherigen abhängt und Tomasulo das Writeback mit Forwarding nicht vor Ausführung der nächsten Instruction nötig hätte
WAR_WAW.txt	7	0.428	Hier werden 2 Takte mehr benötigt, da meine Implementation verschiedene FUs für LOAD und ADD/SUB hat und diese Befehle somit gleichzeitig ausgeführt werden können. Ohne diese parallele Ausführung müssen die zweit Latenztakete des LOAD Befehls addiert werden
LOAD_STORE.txt	7	0.285714	Kein Unterschied zu Tomasulo

Table 1: a. Vergleich mit In-Order-Pipeline ohne dynamisches Scheduling

3.2 b. Variierung der Anzahl der Reservierungsstationen pro FU

Scriptname	Anzahl RS	IPC	Stalltakete	Strukturelle Stalls
NoDependency.txt	2	0.384615	7	2
	8	0.384615	8	0
RAW.txt	2	0.2222	4	2
	8	0.2222	4	0
StructuralStall.txt	2	0.186047	34	25
	8	0.186047	112	0
WAR_WAW.txt	2	0.6	0	1
	8	0.6	0	0
LOAD_STORE.txt	2	0.285714	1	0
	8	0.285714	1	0

Table 2: b. Variierung der Anzahl der Reservierungsstationen pro FU

Die Variation der Anzahl an Reservierungsstationen zeigt unterschiedliche Auswirkungen auf die Programmausführung. Bei `NoDependency.txt` und `RAW.txt` bleibt die IPC un-

verändert. Mit nur zwei Reservation Stations entstehen jedoch zusätzliche strukturelle Stalls, da die RS schneller ausgelastet sind.

Bei `StructuralStall.txt` zeigt sich ein stärkerer Effekt. Mit zwei Reservation Stations treten viele strukturelle Stalls auf, während bei acht Reservation Stations deutlich mehr Instruktionen gleichzeitig warten. Aufgrund der hohen DIV-Latenzen erhöht sich dadurch die Anzahl der Stallzyklen erheblich.

Die Programme `WAR_WAW.txt` und `LOAD_STORE.txt` zeigen kaum Unterschiede zwischen den Konfigurationen. Aufgrund der kurzen Programmlänge und der hohen Parallelität bleibt die Leistung nahezu identisch.

Insgesamt reduzieren zusätzliche Reservation Stations zwar strukturelle Stalls, führen jedoch nicht zwangsläufig zu einer höheren Leistung. Besonders bei langen Latenzen können sich mehr wartende Instruktionen im System ansammeln und zusätzliche Stallzyklen verursachen.

3.3 c. Variierung der Latenzen der Funktionseinheiten

Bei diesem Experiment soll untersucht werden, wie sich das Verhältnis zwischen Stallzyklen und Parallelität verhält, wenn die Latenzen der FUs erhöht werden. Hierfür wird das "WAR_WAW.txt" Script verwendet, da es Parallelität zwischen den beiden FUs gut veranschaulicht.

Latenz LOAD	Latenz ADD/SUB	Stalltakte	Strukturelle Stalls
2	1	0	0
2	2	1	0
2	4	3	0
4	1	0	0

Table 3: c. Variierung der Latenzen der Funktionseinheiten

Bei einer kurzen LOAD-Latenz von 2 und einer ADD/SUB-Latenz von 1 (Standardkonfiguration) ist vollständige Parallelität zwischen LOAD und den ALU-Operationen möglich, ohne dass Stalls entstehen.

Wird die ALU-Latenz auf 2 erhöht, bleibt die Parallelität zwischen LOAD und ADD erhalten, jedoch treten bereits erste Stallzyklen auf. Bei einer weiteren Erhöhung der ALU-FU-Latenz auf 4 bleibt die Parallelität grundsätzlich bestehen, allerdings führen die längeren Ausführungszeiten der ALU-FU zu zusätzlichen Stalls innerhalb der Funktionseinheit.

Bei einer höheren LOAD-Latenz von 4 und einer kurzen ALU-Latenz von 1 verschiebt sich das Verhältnis der Ausführungszeiten. In diesem Fall ist weiterhin Parallelität zwischen LOAD und ALU-Operationen möglich, jedoch dominiert nun die Speicherlatenz die Gesamtausführung.

Insgesamt zeigt sich, dass nicht die absolute Latenz entscheidend ist, sondern das Verhältnis zwischen den Latenzen der einzelnen Funktionseinheiten, da dieses direkt die Parallelität und das Auftreten von Stallzyklen beeinflusst.

3.4 d. Anteil der aufgelösten Abhängigkeiten

Hierfür verwende ich folgendes Programm, welches eine RAW Abhängigkeit und parallele Ausführung enthält:

1	V
2	DIV R0, R1, R1
3	LOAD R2, 0(R0)

In diesem Programm wird genau ein RAW-Hazard transparent überbrückt. Es wird der Stall-zyklus, welcher bei dem Writeback von DIV R0 entstehen würde übersprungen und der Wert kann direkt nach Abschluss des DIV von LOAD durch den CDB verwendet werden. Dennoch gibt es 5 Stalls, welche nicht vermieden werden können, da DIV eine Latenz von 5 Zyklen hat und LOAD somit erst nach diesen 5 Zyklen starten kann, obwohl es ohne die Abhängigkeit sogar parallel laufen könnte.

Der Anteil zwischen aufgelösten und echten Stalls hängt also stark von der Latenz des Befehls ab, welcher den Wert für den nächsten Befehl liefert. Minimal kann das Verhältnis somit 1 zu 1 sein.

3.5 Anfertigungserklärung und KI-nutzung

Hiermit versichere ich, dass ich die vorgelegte Arbeit selbstständig und ohne die Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe. Ich übernehme als Autor beim Einsatz von IT-/KI-gestützten Schreibwerkzeugen die Verantwortung für die inhaltliche Richtigkeit.

Im Rahmen dieser Arbeit wurden Claude (Anthropic) und ChatGPT (OpenAI) unterstützend eingesetzt. Die KI-Nutzung beschränkte sich auf die sprachliche Überarbeitung eigens verfasster Textpassagen sowie die Beantwortung von Verständnisfragen zur Tomasulo-Architektur. Sämtliche Inhalte, Analysen, Code und Ergebnisse wurden eigenständig erarbeitet und inhaltlich überprüft.